



M.3011.002.083 · PEJU/AND/ADA

Desarrollo de Aplicaciones Web con Inteligencia Artificial

Vue

Alex Martín

21 - 29 abril de 2026



Cofinanciado por
la Unión Europea



Fondos Europeos



Sistema Nacional de
Garantía Juvenil

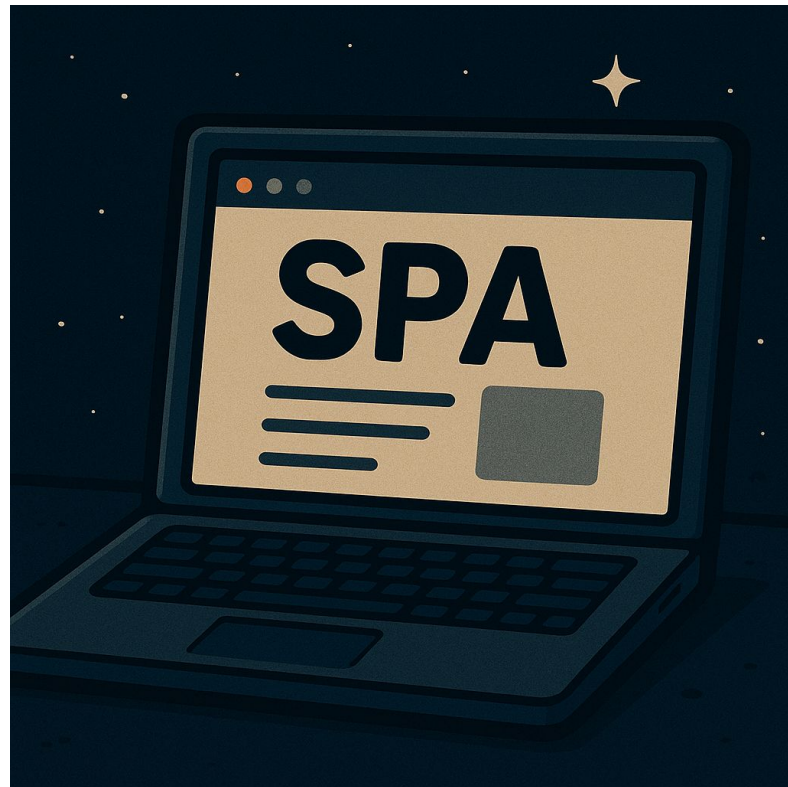


Introducción al desarrollo web moderno



¿Qué caracteriza a una aplicación web moderna?

- Carga y responde de forma muy rápida
- Mejora la experiencia de usuario (como usar una app móvil)
- Se actualiza sin recargar la página
- Separa claramente la parte visual (frontend) de la lógica de datos (backend)
- Permite una navegación más fluida entre secciones, sin saltos de pantalla





Ventajas de una aplicación web moderna

- ➔ **Rapidez** — Carga solo una vez y actualiza sin recargar
- ➔ **Experiencia** — Navegación fluida y sin cortes
- ➔ **Estructura** — Separación clara entre frontend y backend
- ➔ **Reactividad** — Cambios en pantalla sin recargar





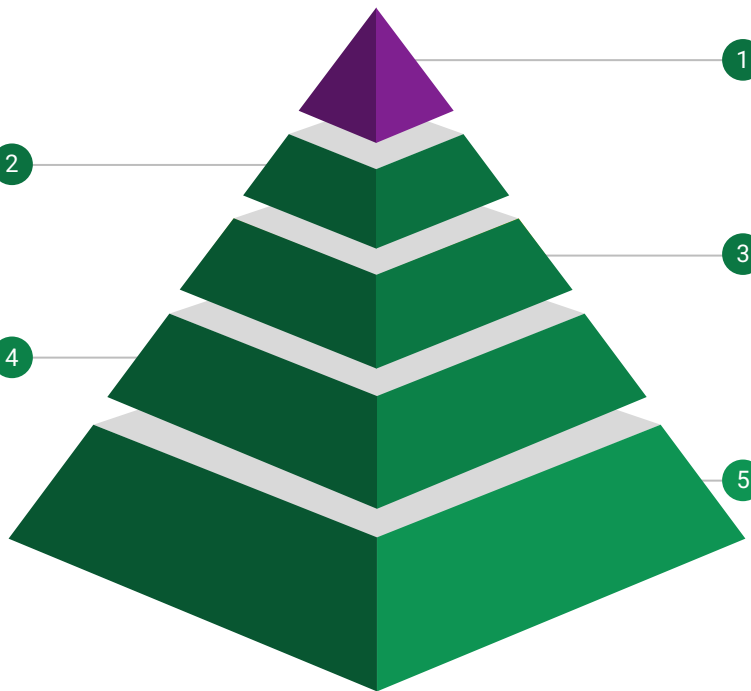
Partes de una Web Application

Código backend

Es el código que se conecta a la base de datos y expone estos datos a la aplicación de cliente. Los lenguajes de backend suelen ser PHP, Python, NodeJs

Base de datos

Es donde almacenamos la información de nuestras aplicaciones web. Las bases de datos pueden ser MySQL, PostgreSQL, MongoDB, Redis, etc ...



Código frontend

Es el código que se ejecuta en el cliente (navegador). La tecnologías aquí son HTML, CSS y Javascript principalmente

Servidor web

Es el software encargado de exponer nuestra aplicación al mundo. Los servidores suelen ser Apache, Nginx e incluso NodeJs

Sistema operativo

Cualquier aplicación debe funcionar sobre un sistema operativo. En aplicaciones web los servidores suelen estar bajo Linux

FRAMEWORKS



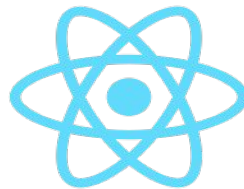
¿Qué es un framework?

Un framework es una herramienta que nos ayuda a crear aplicaciones web más rápido y de forma organizada.

¿Por qué usar uno?

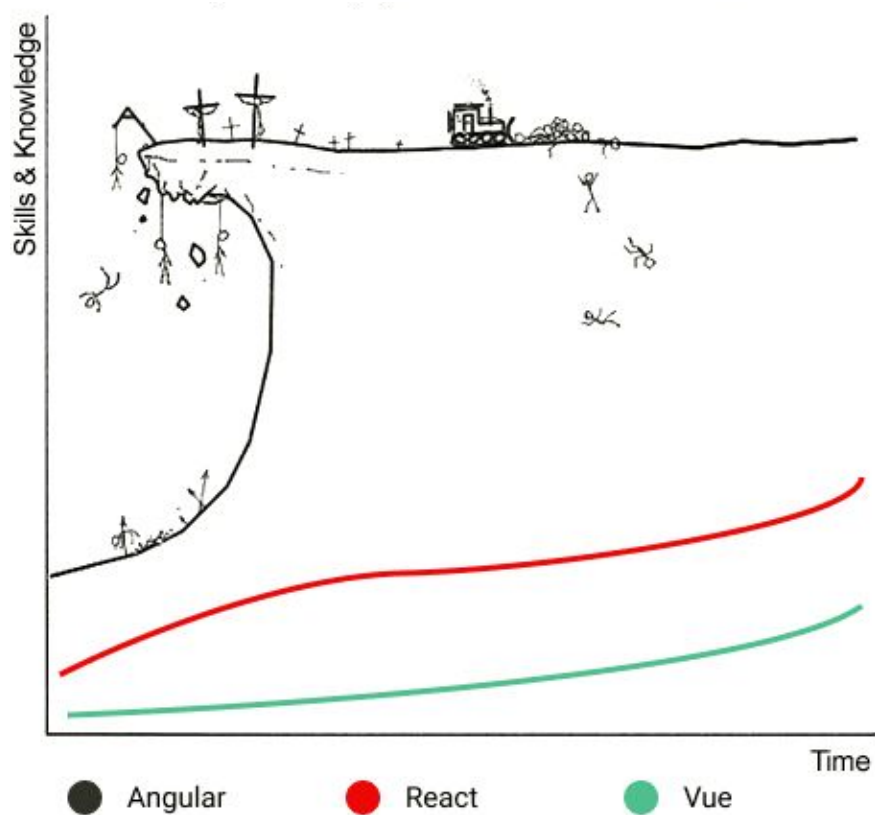
- ➔ Nos da una estructura clara para trabajar
- ➔ Facilita tareas repetitivas (mostrar datos, reaccionar a eventos...)
- ➔ Mejora el mantenimiento del código

 Vue, React y Angular son algunos de los frameworks más populares





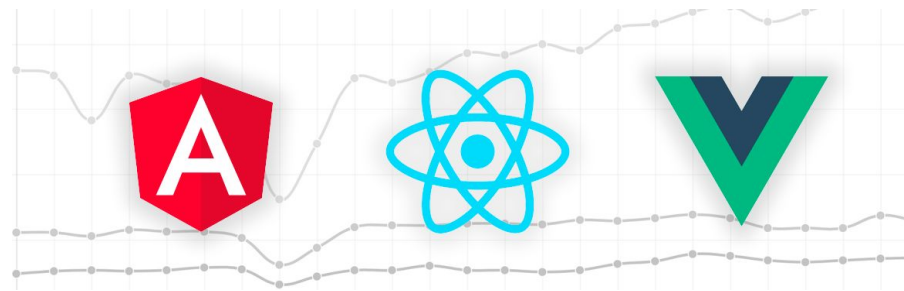
Frameworks. Curva de aprendizaje





Frameworks. Similitudes

- Son open source
- Tienen un crecimiento y mantenimiento constante
- Tienen un CLI para iniciar configurar proyecto base
- Están basados en el concepto de componentes
- Todas son usadas por **big players**:
 - React => Facebook
 - Vue => Alibaba
 - Angular => Microsoft





Frameworks. Diferencias (Angular)

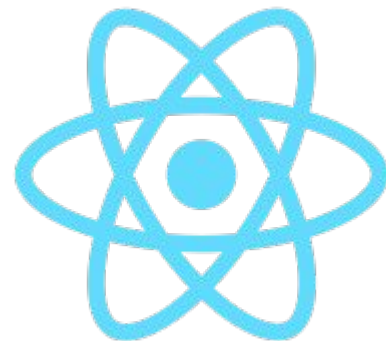
- Angular es un framework completo. Es una evolución de AngularJS de Google.
- Angular usa por defecto Typescript
- En entornos enterprise es muy usado
- Fácil implementar aplicaciones móviles híbridas con Ionic o nativas con NativeScript





Frameworks. Diferencias (React)

- Es una librería para construir interfaces de usuarios
- Es la biblioteca más popular
- Puede expandirse mediante el uso de otras librerías (routing, redux, http lib)
- El núcleo es muy pequeño
- Usa JSX para las vistas
- Podemos desarrollar aplicaciones nativas con React Native





Frameworks. Diferencias (Vue)

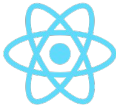
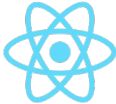
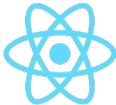
- Es un framework progresivo que crece con complementos y módulos
- Es realmente flexible y productivo
- Curva de aprendizaje pequeña
- Usa un sistema de templating simple separando CSS, HTML y Javascript
- Permite crear aplicaciones móviles con NativeScript, VueNative y Weex





Frameworks. ¿Cuál usar?

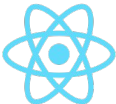
Según requerimientos

Manejo de gran volumen de datos			
Uso exclusivo de vistas			
Manejo de muchas rutas			
Consumo de APIs			



Frameworks. ¿Cuál usar?

Según tipología proyecto

Proyecto personal	¡El que aún no conozcas!
Startups	¡ El que conoces bien !
Cliente pequeño	Según los requisitos
Cliente potencial	 
Cliente grande	 *

Vue.js





VUE

Introducción



¿Qué es Vue?

Vue.js es un framework de JavaScript que nos ayuda a construir interfaces web de forma sencilla y rápida.

- **Fácil de aprender** y usar
- Se enfoca en la **parte visual** de una web
- Permite dividir tu aplicación en **componentes reutilizables**
- Actualiza automáticamente lo que cambia en la pantalla (**reactivo**)
- Ideal para crear **aplicaciones modernas** sin recargar la página (SPA)





¿Cómo se estructura?

Vue organiza el desarrollo en componentes, pequeñas piezas de interfaz que funcionan como bloques reutilizables.

Esta forma de trabajar hace que el código sea más claro y modular.

- Tiene su propio **HTML, CSS y JavaScript**
- Representa una parte visual (como un botón, una tarjeta, etc.)
- Se puede reutilizar en varias partes del proyecto
- Facilita trabajar por secciones en lugar de en un solo archivo





¿Qué hace especial a Vue?

Vue no solo es fácil de usar, también es muy eficiente.

Gracias a su sistema reactivo, cualquier cambio en los datos se refleja automáticamente en la pantalla sin tener que actualizar nada manualmente.

- **Cambia la vista automáticamente** cuando cambian los datos
- Usa **Virtual DOM** para mejorar el rendimiento
- Sigue el **patrón MVVM** para separar datos y presentación
- Tiene una **curva de aprendizaje suave** y progresiva





¿Qué se puede hacer con Vue?

Aunque es sencillo de empezar, Vue es muy versátil.

Se puede usar tanto para sitios pequeños como para aplicaciones grandes, móviles o de escritorio.

- Sitios web **interactivos y dinámicos**
- Aplicaciones completas de una sola página (**SPA**)
- **Apps móviles** con herramientas como Quasar o Vue Native
- **Aplicaciones de escritorio** con Electron





VUE

Utilidades de desarrollo



Vue Devtools

Vue Devtools es una extensión para el navegador que permite inspeccionar el estado interno de una app hecha con Vue.

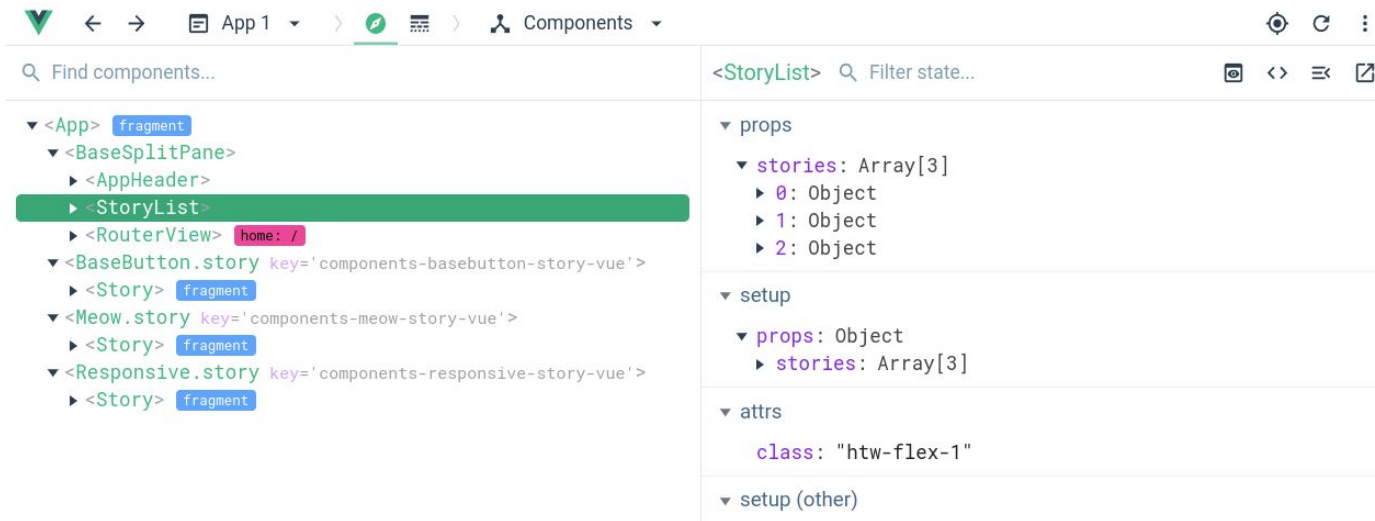
Es muy útil para aprender, depurar y ver cómo se comportan los datos en tiempo real.

- Ver los datos de cada componente en tiempo real
- Inspeccionar la jerarquía de componentes
- Comprobar eventos y propiedades (props)
- Ayuda a entender mejor cómo funciona Vue internamente





Vue Devtools



[Chrome Extension](#)



[Firefox Extension](#)





Extension Visual Studio ([link](#))

Para trabajar cómodamente con Vue en Visual Studio Code, es muy recomendable instalar una extensión que reconozca la sintaxis especial de los archivos .vue.

Esto mejora la experiencia de desarrollo desde el primer día.

- Resalta correctamente el código en HTML, CSS y JS dentro del mismo archivo
- Ofrece autocompletado específico para Vue 3 y Composition API
- Muestra errores y sugerencias en tiempo real
- Facilita el desarrollo con herramientas como `<script setup>` o `<style scoped>`





VUE

Instalación básica



Instalación

➔ Instalación mediante NPM

Desde la consola ejecutando el comando

```
npm create vue@latest
```

Para todo tipo de proyectos

Build basado en VITE

Permite usar SFCs

Permite usar <script setup>

➔ Mediante CDN

Para prototipos o proyectos sencillos

Sin proceso de build

No permite el uso de SFCs

```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```



```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

```
<div id="app">{{ message }}</div>
```

```
<script>
```

```
  const { createApp } = Vue
```

```
  createApp({
```

```
    data() {
```

```
      return {
```

```
        message: 'Hello Vue!'
```

```
      }
```

```
    }
```

```
  }).mount('#app')
```

```
</script>
```



```
<script src="https://unpkg.com/vue@3/dist/vue.global.js"></script>
```

```
<div id="app">{{ message }}</div>
```

```
<script>
```

```
  const { createApp } = Vue
```

```
  createApp({
```

```
    data() {
```

```
      return {
```

```
        message: 'Hello Vue!' ←
```

```
      }
```

```
    }
```

```
  }).mount('#app')
```

```
</script>
```



Estructura básica de un Option Api

```
<script>
export default {
  name: "AppComponent",
  data(){ return {} },
  methods: {},
  watch: {},
  computed:{},
  mounted() {}
}
</script>
```



Estructura básica de un Option Api

```
<script>
```

```
export default {
```

```
  name: "AppComponent",
```



Nombre del componente

```
  data(){ return {} },
```



Variables reactivas

```
  methods: {},
```



Funciones ejecutables

```
  watch: {},
```



Variables observadas

```
  computed: {},
```



Variables “dinámicas”

```
  mounted() {}
```



Hook de montaje

```
}
```

```
</script>
```



VUE

Estilos de uso



Composition API vs Option API

→ Option API

Menos opciones para compartir código en la propia aplicación

```
export default {  
  data() {  
    return {  
      name: 'John',  
    };  
  },  
  methods: {  
    doIt() {  
      console.log("Hello ${this.name}");  
    },  
  },  
  mounted() {  
    this.doIt();  
  },  
};
```

→ Composition API

Más opciones para compartir código de la aplicación

```
export default {  
  setup() {  
    const name = ref('John');  
  
    const doIt = () => console.log("Hello ${name.value}");  
  
    onMounted(() => {  
      doIt();  
    });  
  
    return { name };  
  },  
};
```




Composition API vs Option API

→ **Composition API con <script setup>**

Más conciso

```
<script setup>
  const name = ref('John')
  const doIt = () => console.log(`Hello ${name.value}`)
  onMounted(() => doIt() )
</script>
```



Composition API. El método setup

- En el añadiremos toda la lógica de nuestros componentes.
- **Setup** recibe dos parámetros (props y context) y debe devolver un objeto con las propiedades y métodos que se usarán en *template*
 - **Props** contiene todas propiedades que el componente aceptar
 - **Context** es el contexto de Vue, es análogo al uso de *this* en Vue 2.x

```
setup() {  
  const state = reactive({  
    count: 0,  
    double: computed(() => state.count * 2)  
  })  
  
  function increment() {  
    state.count++  
  }  
  
  return {  
    state,  
    increment  
  }  
}
```



Composition API. El método setup

- En el **setup** añadiremos toda la lógica de nuestros componentes.
- **Setup** recibe dos parámetros (props y context) y debe devolver un objeto con las propiedades y métodos que se usarán en **template**
- **Props** contiene todas las propiedades que el componente acepta
- **Context** es el contexto de Vue, es análogo al uso de **this** en Vue 2.x

```
setup() {  
  const state = reactive({  
    count: 0,  
    double: computed(() => state.count * 2)  
  })  
  
  function increment() {  
    state.count++  
  }  
  
  return {  
    state,  
    increment  
  }  
}
```



VUE

Instalación estándar



Instalación

➔ **Crear un proyecto Vue con NPM** (Nota: Este comando crea el proyecto como composition API)

```
npm create vue@latest
```

```
✓ Project name: ... <your-project-name>
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit testing? ... No / Yes
✓ Add Cypress for both Unit and End-to-End testing? ... No / Yes
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes
```

```
Scaffolding project in ./<your-project-name>...
```

```
Done.
```



VUE Sintaxis



Sintaxis básica

→ Interpolación de texto y HTML

TEMPLATE

```
<p>
Usando interpolación de texto:
{{ text }}
</p>
<p>
  Con directiva v-html:
  <span v-html="html"></span>
</p>
```

SCRIPT

```
<script setup>
  const text = "Hello world"
  const html= "<strong>Hello world</strong>"
</script>
```



Sintaxis básica

→ Condicionales simples

TEMPLATE

```
<div id="app">
  <span v-if="isVisible">Cu cu trá</span>
</div>
```

SCRIPT

```
<script setup>
  const isVisible = true
</script>
```




Sintaxis básica

→ Loops

TEMPLATE

```
<div id="app">
  <ol>
    <li v-for="(todo,index) in todos">
      {{index}} - {{ todo.text }}
    </li>
  </ol>
</div>
```

JS

```
<script setup>
const todos = [
  { text: 'Hola' },
  { text: 'Amigos' }
]
</script>
```



Sintaxis básica

→ Loops (key)

TEMPLATE

```
<div id="app">
  <ol>
    <li v-for="(todo,index) in todos" :key="todo.id">
      {{ todo.text }}
    </li>
  </ol>
</div>
```

SCRIPT

```
<script setup>
const todos = [
  { id: "1", text: 'Hola', visible: true },
  { id: "2", text: 'Amigos', visible: false }
]
</script>
```



Sintaxis básica

→ Interacción con el usuario (v-on y methods)

TEMPLATE

```
<div id="app">
  <button @click="saluda">
    Saluda
  </button>
</div>
```

SCRIPT

```
<script setup>
function saluda () {
  alert("Hola mundo")
}
</script>
```



→ Binding

Para vincular un dato con una atributos HTML usaremos la propiedad ***v-bind:propiedad*** o de forma corta ***:propiedad***

TEMPLATE

```
<div id="app">
  <button :disabled="false">
    Mi Botón
  </button>
</div>
```



VUE

Reactividad



Composition API. Ref

- Ref actúa como wrapper para transformar variables en objetos reactivos.
- En Ref podemos usar primitivos de javascript (string, boolean, number, ...)
- Ref devuelve un objeto con la propiedad value que podemos mutar.

```
import { ref } from 'vue';

export default {
  setup() {
    const count = ref(0);
    const increment = () => {
      count.value++;
    };
    return { count, increment };
  }
};
```



Composition API. Ref

- Ref actúa como wrapper para transformar variables en objetos reactivos.
- En Ref podemos usar primitivos de javascript (string, boolean, number, ...)
- Ref devuelve un objeto con la propiedad value que podemos mutar.

```
import { ref } from 'vue';

export default {
  setup() {
    const count = ref(0);
    const increment = () => {
      count.value++;
    };
    return { count, increment };
  }
};
```



Composition API. Reactive

- Reactive funciona de forma similar a Ref, con la diferencia que las propiedades **no son envueltas** en un objeto como con Ref.
- Reactive es principalmente usado para hacer reactivo un objeto.

```
import { reactive } from 'vue';  
  
const state = reactive({  
  count: 0  
})  
  
state.count++;
```

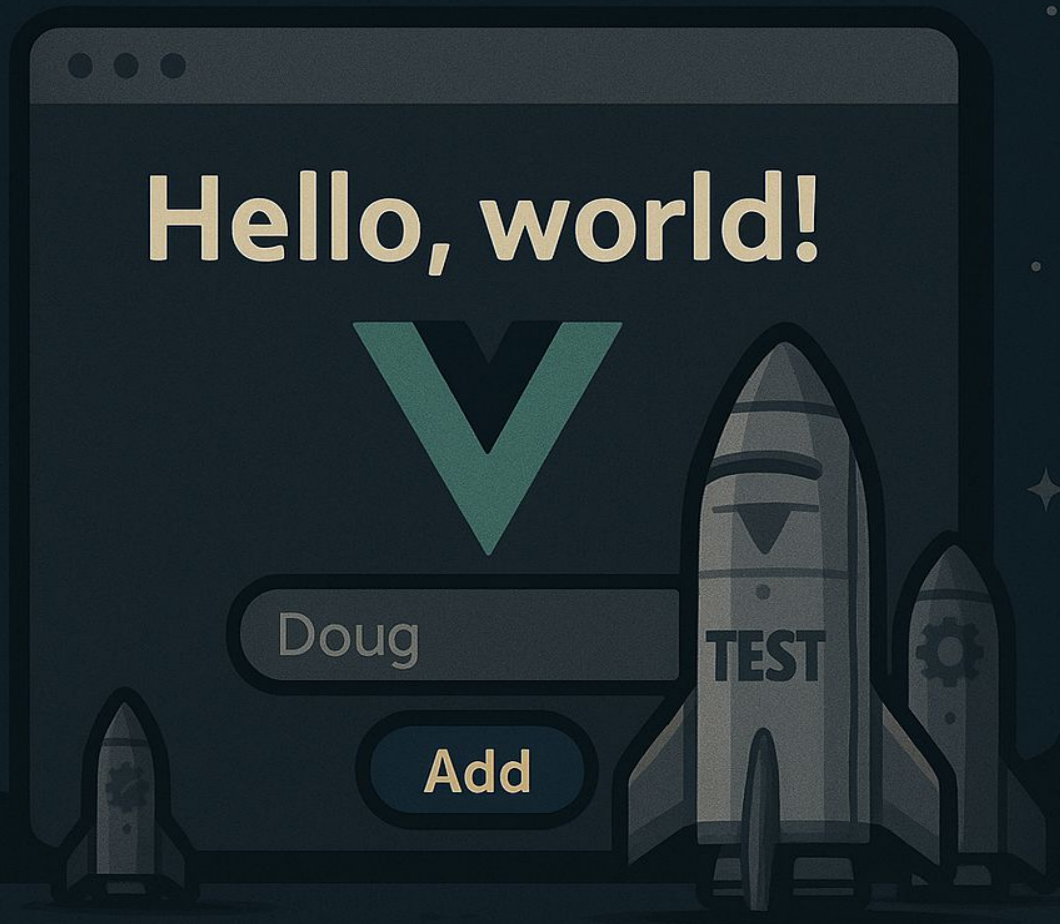

Sintaxis básica

Estás en la cabina de mando de la nave Starcode-17.

Los tripulantes están en hibernación. Tu tarea es activar la consola y mostrar en pantalla quiénes están a bordo. ¡La misión está por comenzar!

Objetivos técnicos:

- Crear un proyecto con `npm create vue@latest`
- Usar `ref` para controlar el estado de visibilidad
- Usar `v-for` para mostrar la lista de tripulantes
- Mostrar un botón que revela la tripulación cuando se pulsa



Sintaxis básica

La Starcode-17 está lista para partir. Tu tarea es activar la consola de tripulación.

Al principio, no se muestra ningún nombre.

Al pulsar un botón añadirá un tripulante a la lista (usa la función `getCrew` proporcionada)

Debes añadir un botón que borre el último tripulante.

```
function getCrew() {  
  const nombres = ['Aisha', 'Leo', 'Ravi',  
    'Tara', 'Ivy', 'Max', 'Lena', 'Alex']  
  const indice = Math.floor(Math.random() *  
nombres.length)  
  return nombres[indice]  
}
```

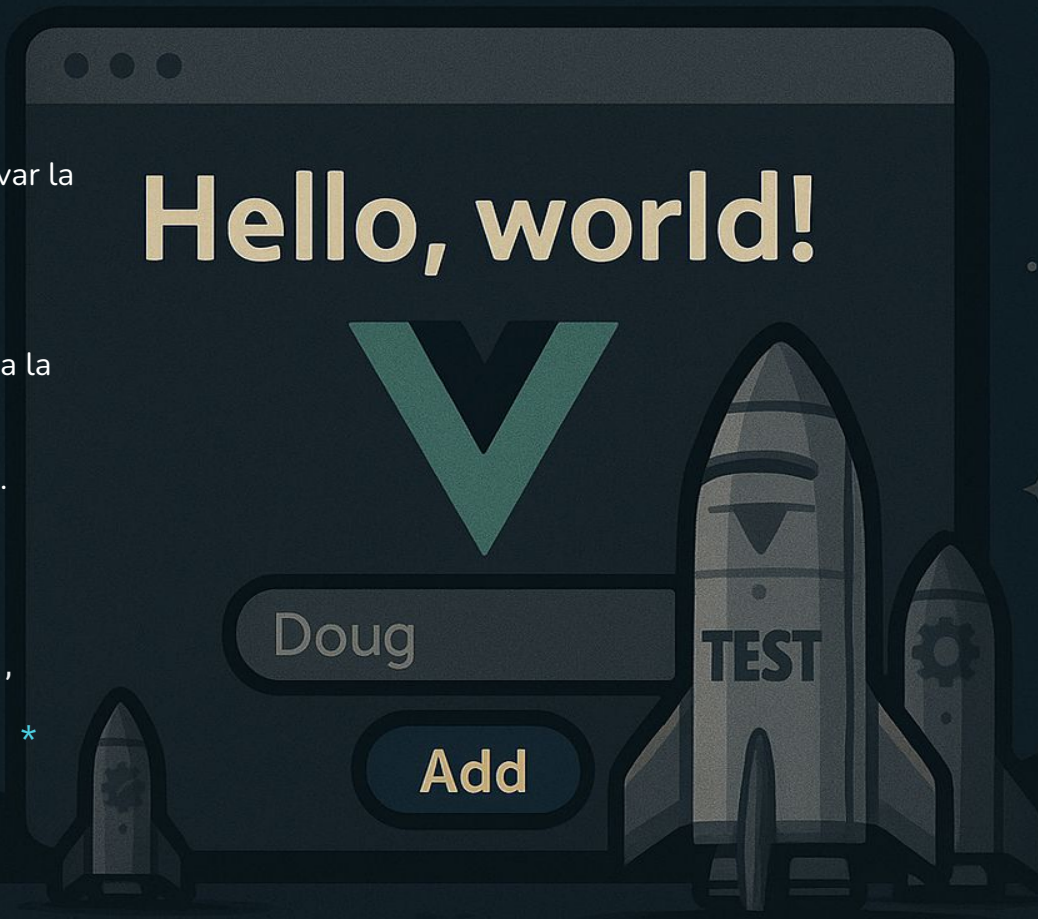
Hello, world!



Doug

Add

TEST





VUE

Renderizado condicional



Renderización condicional

→ Vue provee de varios bloques para aplicar condicionales

◆ Bloque if

```
<h1 v-if="ok">Sí</h1>
```

◆ Bloque else

```
<h1 v-if="ok">Sí</h1>  
<h1 v-else>No</h1>
```

◆ Bloque elseif

```
<div v-if="type === 'A'"> A </div>  
<div v-else-if="type === 'B'"> B </div>  
<div v-else> Si no es A, B o C </div>
```



Renderización condicional

→ Vue provee de varios bloques para aplicar condicionales

◆ Grupos condicionales

```
<template v-if="ok">
  <h1>Título</h1>
  <p>Párrafo 1</p>
  <p>Párrafo 2</p>
</template>
```

◆ v-show

```
<h1 v-show="ok">Hola!</h1>
```

La diferencia entre un v-if y un v-show es que show oculta el elemento mediante css e if lo elimina del DOM



VUE
Bucles



Bucles

→ Vue provee de v-for para recorrer elementos iterables de tipo **array** y **object**

◆ Recorrer un array

```
<ul id="example-1">
  <li v-for="(item, index) in items">
    {{index}} - {{ item.mensaje }}
  </li>
</ul>
```

También es posible usar **of** como delimitador en vez de **in**

```
<div v-for="item of items"></div>
```



Bucles

◆ Recorrer un objeto

```
<ul id="v-for-object" class="demo">
  <li v-for="value in object">
    {{ value }}
  </li>
</ul>
```

También es posible obtener la clave del objeto y el índice de iteración

```
<div v-for="(value, key, index) in object">
  {{ index }} -- {{ key }}: {{ value }}
</div>
```




Bucles

◆ Recorrer un rango

```
<div>
  <span v-for="n in 10">{{ n }} </span>
</div>
```

◆ v-for en un template (agrupación)

```
<ul>
  <template v-for="item in items">
    <li>{{ item.msg }}</li>
    <li class="divider" role="presentation"></li>
  </template>
</ul>
```



La importancia de :key

- ◆ En los bucles con **v-for** debemos usar la clave **:key** para identificar de forma única cada elemento de la lista renderizada.

```
<ul>
  <li v-for="task in tasks" :key="task.id">{{ task.name }}</li>
</ul>
```

- ◆ Si no aplicamos :key, al eliminar un elemento, Vue puede confundir los elementos restantes



Arrays. Detección de cambios

- ◆ Vue detectará cualquier cambio y actualizará la vista al aplicar cambios con los métodos “mutantes”:

push()
pop()
shift()
unshift()
splice()
sort()
reverse()

Aunque existen métodos no mutantes como **filter()**, **concat()** o **slice()**, Vue no renderiza completamente la vista y reutiliza los elementos del DOM



VUE

Propiedades computadas



Propiedades computadas

- Nos permite añadir lógica a variables que usaremos dentro del template.
- Cada propiedad de computed debe ser una función que retorne un objeto primitivo de javascript

HTML

```
<div id="app">
  {{ msg }}
  {{ bigMsg }}
</div>
```

SCRIPT SETUP

```
import { ref, computed } from 'vue';

const msg = ref("Hola");

const bigMsg = computed(() => {
  return msg.value.toUpperCase();
});
```



Setter computado

→ Por defecto las propiedades son solo get, aunque podemos usar un set cuando lo necesitemos

HTML

```
<div id="app">
  {{ firstname }} {{ lastname }}
  {{ fullname }}
</div>
```

JS

```
import { ref, computed } from 'vue';

const firstname = ref("");
const lastname = ref("");

const fullname = computed({
  get: () => {
    return `${firstname.value} ${lastname.value}`;
  },
  set: (newValue) => {
    const names = newValue.split(' ');
    firstname.value = names[0];
    lastname.value = names[names.length - 1];
  }
});
```



VUE Watchers



Watcher

- ➔ Permite observar y reaccionar a cambios en los datos de la aplicación
- ➔ Una función se ejecutará cada vez que el elemento escuchado cambien

TEMPLATE

```
<template>
  <div>
    <input v-model="message" placeholder="Escribe algo">
    <p>{{ message }}</p>
  </div>
</template>
```

SCRIPT SETUP

```
const message = ref('');
watch(message, (newValue, oldValue) => {
  console.log('Nuevo valor:', newValue);
  console.log('Valor anterior:', oldValue);
});
```




Watcher múltiple

TEMPLATE

```
<template>
  <div>
    <input v-model="username" placeholder="Nombre de
usuario">
    <input v-model="email" placeholder="Correo
electrónico">
  </div>
</template>
```

SCRIPT SETUP

```
import { ref, watch } from 'vue';

const username = ref('');
const email = ref('');

watch([username, email], ([newUsername, newEmail]) => {
  console.log('Nuevo nombre de usuario:', newUsername);
  console.log('Nuevo correo electrónico:', newEmail);
});
```



Deep Watcher

- Se utiliza para observar cambios en las propiedades anidadas de un objeto o en los elementos de un array
- Usarlo en exceso podría perjudicar el rendimiento de nuestra aplicación

```
<div>
  <input v-model="user.name" placeholder="Nombre">
  <input v-model="user.age" placeholder="Edad">
</div>
```

```
import { ref, watch } from 'vue';
const user = ref({
  name: '',
  age: ''
});
watch(user, (newValue, oldValue) => {
  console.log('Nuevo valor de usuario:',
newValue);
  console.log('Valor de usuario anterior:',
oldValue);
}, { deep: true });
```



VUE

Template ref



Template Ref

- Permiten acceder a elementos del DOM o componentes hijos directamente desde el script.
- Útiles para manipular el DOM de manera imperativa.

```
<template>
  <input ref="myInput" />
  <ChildComponent ref="childComponentRef" />
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';

const myInput = ref(null);
const childComponentRef = ref(null);

onMounted(() => {
  console.log(myInput.value);
  console.log(childComponentRef.value);
});
</script>
```



Template Ref

- Permiten acceder a elementos del DOM o componentes hijos directamente desde el script.
- Útiles para manipular el DOM de manera imperativa.

```
<template>
  <input ref="myInput" />
  <ChildComponent ref="childComponentRef" />
</template>
```

```
<script setup>
import { ref, onMounted } from 'vue';

const myInput = ref(null);
const childComponentRef = ref(null);

onMounted(() => {
  console.log(myInput.value);
  console.log(childComponentRef.value);
});
</script>
```



VUE

Manipuladores clases



Manipuladores de clases

- Vue nos permite modificar los atributos de clases de un elemento HTML con varias sintaxis disponibles:

◆ Sintaxis de objeto

```
<div :class="{ active: isActive }"></div>  
<div class="static" :class="{ active: isActive, 'text-danger': hasError }"></div>  
<div :class="classObject"></div>
```

◆ Sintaxis de array

```
<div :class="[activeClass, errorClass]"></div>  
<div :class="[isActive ? activeClass : '', errorClass]"></div>
```



Manipuladores de estilos

➔ Al igual que con los manipuladores de clases, con los estilos también podemos usar varias sintaxis:

◆ Sintaxis de objeto

```
<div :style="{ color: activeColor, fontSize: fontSize + 'px' }"></div>
<div :style="styleObject"></div>
```

◆ Sintaxis de array

```
<div v-bind:style="[baseStyles, overridingStyles]"></div>
```

* Cuando usemos propiedades que posean prefijos de operadores (por ejemplo transform). Vue se los asignará automáticamente (-webkit-transform, -ms-transform) [[Más sobre usos de prefijo de operadores](#)]



VUE Eventos



Eventos

- ◆ Para capturar un evento usaremos la directiva ***v-on:nombreevento*** o **@** en su forma abreviada

```
<div id="example-1">  
  <button @click="counter += 1">Add 1</button>  
  <p>Se ha hecho clic en el botón de arriba {{ counter }} veces.</p>  
</div>
```

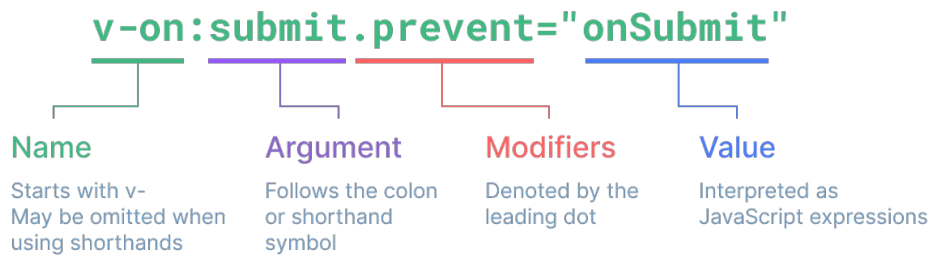
- ◆ Modificadores de eventos

```
<a @click.stop="hasEsto"></a>  
<form @submit.prevent="onSubmit"></form>
```

```
<a @click.once="hasEsto"></a>
```



Eventos



◆ Modificadores de eventos

```
<a @click.stop="hasEsto"></a>  
<form @submit.prevent="onSubmit"></form>
```

```
<a @click.once="hasEsto"></a>
```



Eventos



Modificadores de teclas

```
<input v-on:keyup.13="submit">
<input v-on:keyup.enter="submit">
<input v-on:keyup.f1="submit">
```

Lista completa de alias:

- .enter
- .tab
- .delete
- .esc
- .space
- .up
- .down
- .left
- .right
- .ctrl
- .alt
- .shift
- .meta



Binding en formularios (two-way-binding)

- ➔ Mediante la directiva **v-model** podremos vincular el valor de un campo de introducción por parte del usuario con una propiedad. Las propiedades bindadas con v-model **deben** existir en data

```
<input v-model="message">

<input type="checkbox" id="checkbox" v-model="checked">

<input type="radio" id="uno" value="Uno" v-model="picked">
<input type="radio" id="Dos" value="Dos" v-model="picked">

<select v-model="selected">
  <option v-for="option in options" :value="option.value">
    {{option.text}}
  </option>
</select>
```



Binding en formularios (two-way-binding)

◆ Modificadores

```
<input v-model.number="age" type="number">
```

```
<input v-model.trim="msg">
```



VUE

Directivas personalizadas



Directivas personalizadas. ¿Qué son?

- Las directivas personalizadas en Vue 3 permiten crear comportamientos personalizados que se pueden aplicar directamente a los elementos del DOM.
- Reaccionan a una serie de hooks para manejar eventos del DOM:
beforeMount, mounted, beforeUpdate, updated, beforeUnmount, unmounted
- Usaremos el prefijo **v-nombredirectiva** para usarla en nuestro template.
- Cada directiva debe configurarse en el fichero main.js de nuestra aplicación

```
import { createApp } from 'vue';
import App from './App.vue';
import colorDirective from './directives/colorDirective';

const app = createApp(App);
app.directive('color', colorDirective);
app.mount('#app');
```

```
<template>
  <div>
    <p v-color="'red'">Hover over this</p>
  </div>
</template>
```




Directivas personalizadas. ¿Qué son?

- Las directivas personalizadas en Vue 3 permiten crear comportamientos personalizados que se pueden aplicar directamente a los elementos del DOM.
- Reaccionan a una serie de hooks para manejar eventos del DOM:
beforeMount, mounted, beforeUpdate, updated, beforeUnmount, unmounted
- Usaremos el prefijo **v-nombredirectiva** para usarla en nuestro template.
- Cada directiva debe configurarse en el fichero main.js de nuestra aplicación

```
import { createApp } from 'vue';  
import App from './App.vue';  
import colorDirective from './directives/colorDirective';  
  
const app = createApp(App);  
app.directive('color', colorDirective);  
app.mount('#app');
```

```
<template>  
  <div>  
    <p v-color="'red'">Hover over this</p>  
  </div>  
</template>
```



Directivas personalizadas. ¿Qué son?

```
export default {  
  beforeMount(el, binding) {  
    const originalColor = el.style.color;  
    const hoverColor = binding.value || 'blue';  
  
    el.addEventListener('mouseenter', () => {  
      el.style.color = hoverColor;  
    });  
  
    el.addEventListener('mouseleave', () => {  
      el.style.color = originalColor;  
    });  
  },  
  unmounted(el) {  
    el.removeEventListener('mouseenter', this.mouseEnterHandler);  
    el.removeEventListener('mouseleave', this.mouseLeaveHandler);  
  }  
};
```



Práctica:

El tiempo de Maldonado (vue edition)

Rescata la práctica realizada realizada con vanilla js y pásala a Vue.

Organízate de la siguiente manera:

- Crea un proyecto con NPM
- Déjalo limpio para poder trabajar.
- Abre el proyecto de maldonado y extrae los elementos necesario para que funcione sin javascript (solo la vista), añadiendo todo el HTML en el componente principal de Vue (app.vue)
- Añade la lógica javascript de tu práctica ajustando lo necesario para que funcione dentro de VUE





Práctica:

La lista de la compra

Realizar un formulario que permita añadir productos un listado de productos

Cada producto tendrán los siguientes atributos:

- Identificador
- Nombre
- Precio

Requisitos

Cada elemento de la lista debe poder **editarse** y **eliminarse**

Bonus

Debe contener un **computed** que calcule el el precio total de los elementos añadidos a la lista

Añadir categorías a los productos y permitir filtrar por ellas



TIPS



Usa esta función para generar identificadores aleatorios

```
function generateId() {  
  return Math.random().toString(36).substr(2, 8);  
}
```



VUE Componentes



Importar y usar un componente

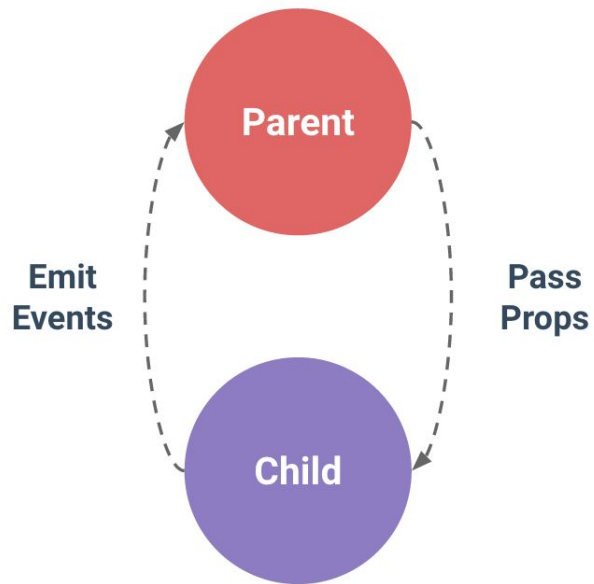
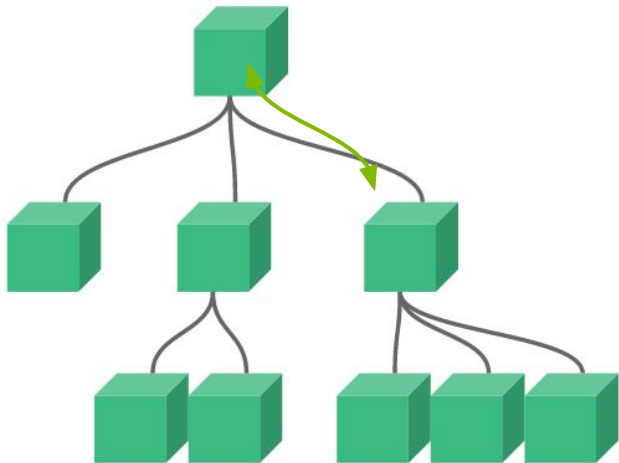
- Para usar un componente dentro de otro deberemos importarlo y registrarlo
- Se aconseja añadir todos los componentes en un directorio **components**
- Intenta que tus componentes tengan nombres compuestos

```
<template>
  <div>
    
    <HelloWorld />
    <HelloWorld />
  </div>
</template>
<script setup>
import HelloWorld from '@components/HelloWorld.vue'
</script>
```



Comunicación entre componentes

→ Propiedades y eventos





Propiedades. Definición

- Parámetros pasados a un componente para personalizar su comportamiento o apariencia.
- Las propiedades no pueden manipularse directamente desde el hijo.
- Facilitan la reutilización de componentes..

```
<template>
  <div>
    <p>{{ message }}</p>
    <p>{{ count }}</p>
  </div>
</template>
```

```
<script setup>
import { defineProps } from 'vue';

const props = defineProps({
  message: String,
  count: {
    type: Number,
    default: 0
  }
});
</script>
```




Propiedades. Paso de propiedades

- Se usa la notación de atributos para pasar propiedades desde el padre
- Los tipos de datos pasados pueden ser String, Number, Boolean, Array, Object, Function o Symbol
- Una prop puede ser de varios tipos, definiendola como un array. Ej: myProp: [String, Number]

```
<template>
  <ChildComponent message="Hello, Vue!" :count="5" />
</template>

<script setup>
import ChildComponent from './ChildComponent.vue';
</script>
```



Eventos. Definición

- ➔ Permite comunicar cambios o interacciones desde un componente hijo a un componente padre.
- ➔ Facilita la interacción entre componentes.

```
<template>
  <button @customEvent="handleClick">Click
  Me</button>
</template>
```

```
import { defineEmits } from 'vue';
const emit = defineEmits(['customEvent']);
const handleClick = () => {
  emit('customEvent', 'Hello, Parent!');
};
```



Componentes dinámicos. Definición

- ➔ Permite renderizar un componente u otro mediante el uso de el componente `<component>`
- ➔ Mediante el atributo especial `:is` podemos definir que componente mostrar

```
<template>
  <component :is="currentComponent"/>
</template>
```

```
import Componente1 from '@/components/Componente1';
import Componente2 from '@/components/Componente2';

const isFirst = ref(false);
const currentComponent = isFirst.value? Componente1:
Componente2
```



VUE

Custom v-model



v-model personalizado

➔ Desde la versión 3.4 podemos crear fácilmente nuestros propios v-model

```
<script setup>
const model = defineModel({default:0})

function update() {
  model.value++
}
</script>

<template>
  <div>v-model es: {{ model }}</div>
</template>
```



Práctica:

Todo List

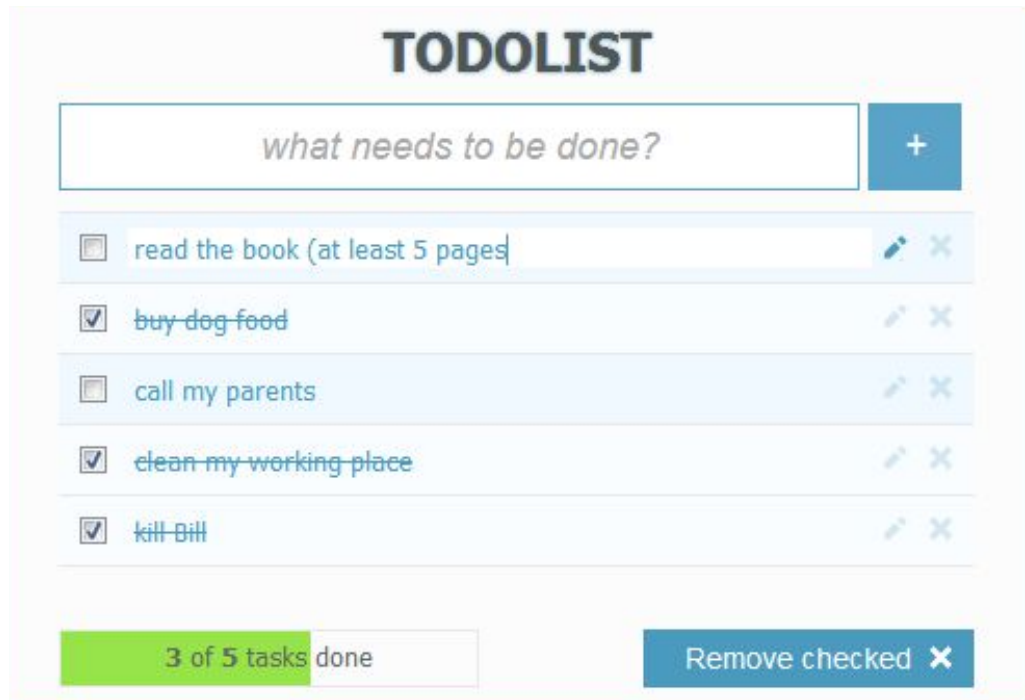
Desarrollar una aplicación de ToDo List.

La aplicación debe permitir a los usuarios agregar, editar, y eliminar tareas.

Además, las tareas deben poder ser marcadas como completadas.

La aplicación debe estar organizada en múltiples componentes para promover una estructura limpia y mantenible, haciendo uso de props, eventos y v-model.

Puedes basar tu diseño en la imagen que se muestra a continuación.





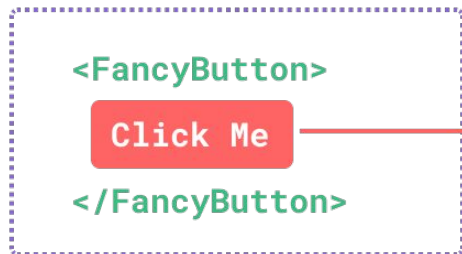
VUE
Slots



Slots. Definición

- Un slot es un espacio reservado en un componente donde se puede insertar contenido dinámico.
- Permite crear componentes más flexibles y reutilizables al aceptar contenido externo
- Existen tres tipos: Slots por defecto, Slots nombrados y Scope Slots

parent template



replace

<FancyButton> template



slot content



slot outlet



Slots. Por defecto y named

```
<div class="container">
  <header>
    <slot name="header">Contenido por defecto</slot> ← Named
  </header>
  <main>
    <slot>Contenido por defecto</slot> ← Por defecto
  </main>
</div>
```

```
<MyComponente>
Información que va al slot por defecto
<template #header>Esto va al slot header</template>
</MyComponente>
```



Slots. Scoped

<MyComponent> template

```
<div>
```

```
  <slot
```

```
    :text="greetingMessage"
```

```
    :count="1"
```

```
  />
```

```
</div>
```

slot props

parent template

```
<MyComponent v-slot="slotProps">
```

```
  {{ slotProps.text }}
```

```
  {{ slotProps.count }}
```

```
</MyComponent>
```

● scoped slot content

● scoped slot outlet

```
<div>
```

```
  <slot :text="greetingMessage" :count="1"></slot>
```

```
</div>
```

```
<MyComponent v-slot="slotProps">
```

```
  {{ slotProps.text }} {{ slotProps.count }}
```

```
</MyComponent>
```

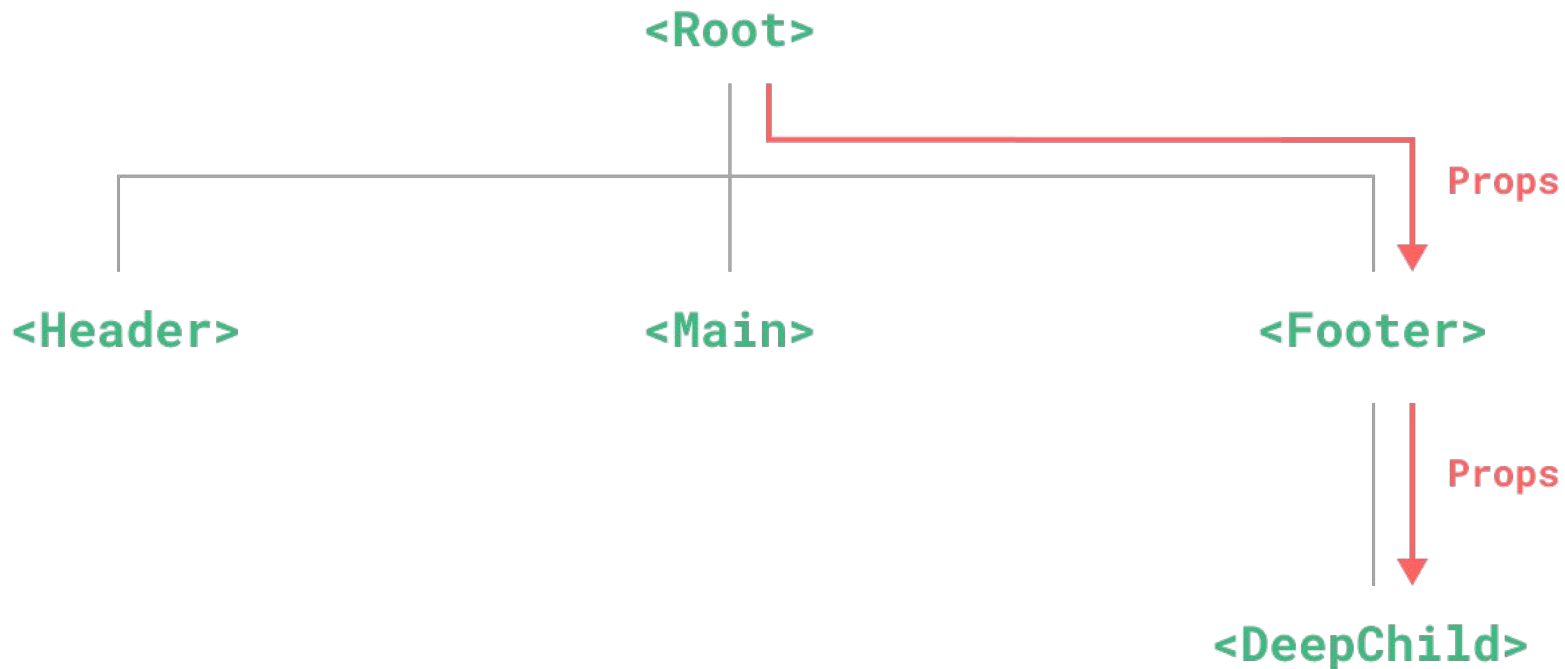


VUE

Provide / Inject

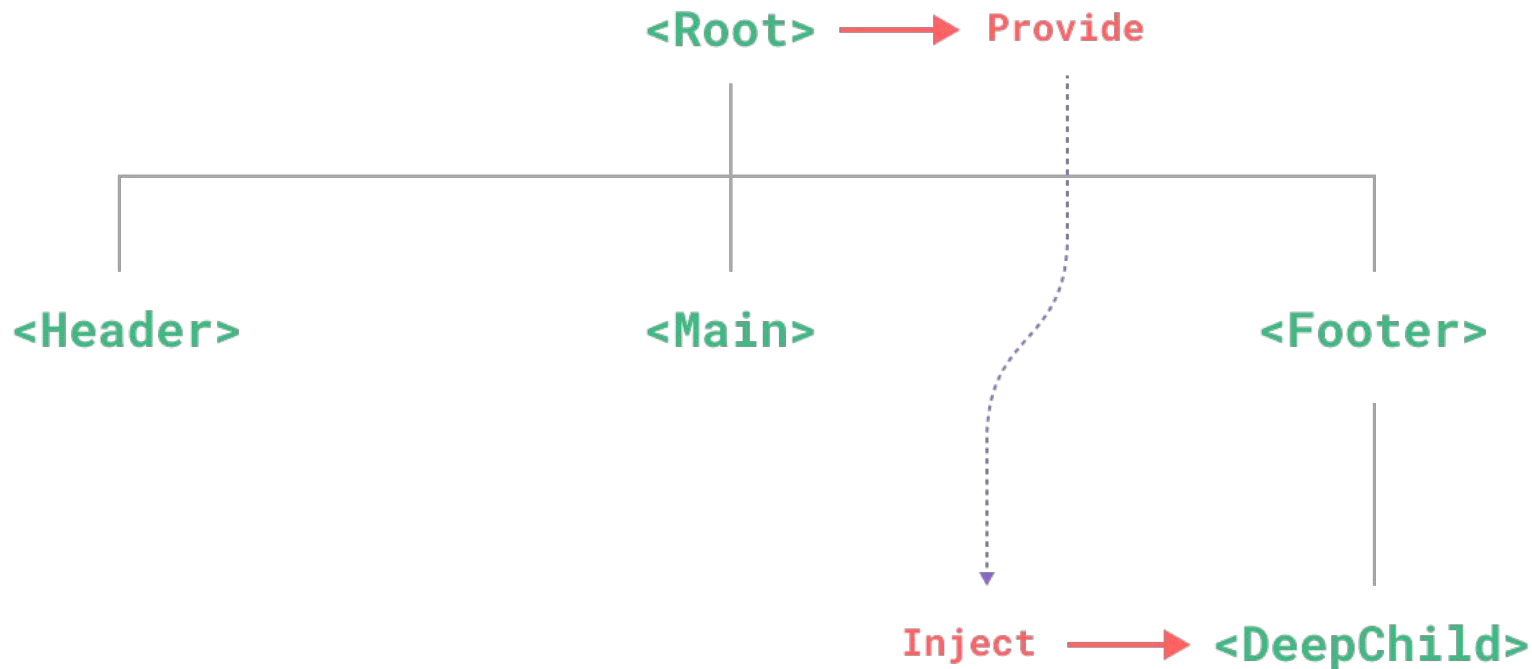


Props drilling. Complejidad





Props drilling. Resolución con provide / inject





Provide / Inject. Ventajas y desventajas

- ➔ Reducción de Props Drilling
- ➔ Mejora de la Mantenibilidad
- ➔ Flexibilidad
- ➔ Depuración compleja
- ➔ Dependencia implícita

```
<template>
  <ParentComponent />
</template>

<script setup>
import { provide } from 'vue';

const message = 'Hello from App';
provide('message', message);
</script>
```

```
<template>
  <p>{{ message }}</p>
</template>

<script setup>
import { inject } from 'vue';

const message = inject('message');
</script>
```



VUE

Ciclos de vida



Componentes. Ciclo de vida (Options API)

→ Los componentes tienen un ciclo de vida basado en 4 fases:

- ◆ Creación
- ◆ Montaje
- ◆ Actualización
- ◆ Desmontaje

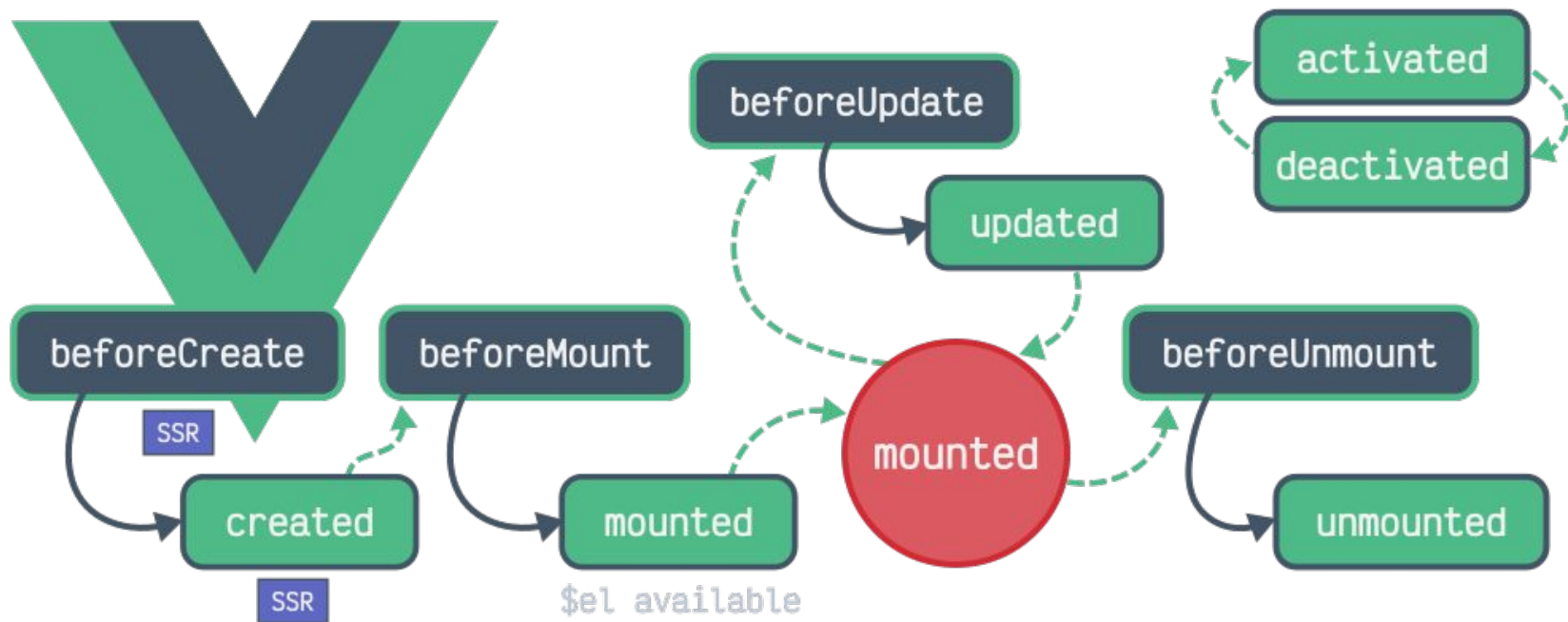
Cada fase tiene dos hooks antes y después de pasar por cada fase, por lo que tenemos 8 momentos donde podemos ejecutar funcionalidades.

Estos hooks son:

- | | |
|------------------|-----------------|
| - beforeCreate | - beforeUpdate |
| - created | - updated |
| - beforeMount | - beforeUnmount |
| - mounted | - unmounted |



Componentes. Ciclo de vida





Componentes. Hooks

Option API	Composition API	¿Cuándo se llama?
beforeCreate	Se ejecuta en el setup	Antes de que se inicialice el componente y antes de que se configuren las dependencias reactivas.
created	Se ejecuta en el setup	Después de que se ha creado el componente y se han configurado las dependencias reactivas.
beforeMount	onBeforeMount	Antes de que el componente sea montado en el DOM.
mounted	onMounted	Después de que el componente ha sido montado en el DOM
updated	onUpdated	Cuando los datos reactivos del componente se han actualizado y el DOM ha sido vuelto a renderizar.
beforeUnmount	onBeforeUnmount	Se llama justo antes de que el componente sea desmontado del DOM.
unmounted	onUnmounted	Después de que el componente ha sido desmontado del DOM



VUE

Composables



Composables. ¿Qué son?

- Funciones reutilizables que encapsulan y organizan la lógica reactiva en aplicaciones Vue.
- Facilitan la reutilización de código y mejoran la separación de responsabilidades en la aplicación.
- Como convención, los composables suelen diferenciarse de otras funciones por el uso del prefijo “**use**” delante del nombre de la función. Ejemplo `useCounter()`

Ventajas

- **Reutilización de código.** Podemos encapsular la lógica común para usar en varios componentes.
- **Organización y mantenimiento.** Ayuda a organizar el código de manera más limpia y modular
- **Separación de preocupaciones.** Podemos separar la lógica de la aplicación y la de la interfaz.



Ejemplo de composable

useCounter.js

```
// useCounter.js
import { ref } from 'vue';

export function useCounter() {
  const count = ref(0);

  const increment = () => {
    count.value++;
  };

  const decrement = () => {
    count.value--;
  };

  return {
    count,
    increment,
    decrement
  };
}
```

```
<template>
  <div>
    <p>Count: {{ count }}</p>
    <button @click="increment">Increment</button>
    <button @click="decrement">Decrement</button>
  </div>
</template>

<script setup>
import { useCounter } from './useCounter';
const { count, increment, decrement } =
useCounter();
</script>
```



Composables. VueUse

- VueUse es una colección de composables utilitarios basados en Vue 3 que proporcionan funciones reactivas listas para usar.
- Entre estas funciones podemos encontrar algunas interesantes como:
 - useLocalStorage:** Permite manipular fácilmente localStorage
 - useMouse:** Controla la posición del ratón
 - useClipboard:** Permite responder a eventos de portapapeles.

Instalación

```
npm install @vueuse/core
```



Práctica:

Extendiendo Todo List

Añade persistencia de datos mediante localStorage y el composable useStorage de VueUse

TODOLIST

what needs to be done?



read the book (at least 5 pages)



~~buy dog food~~



call my parents



~~clean my working place~~



~~kill Bill~~



3 of 5 tasks done

Remove checked 



VUE

Vue Router



Vue Router. ¿Qué es?

- ➔ Vue Router es la librería oficial de Vue.js para manejar el enrutamiento de aplicaciones de una sola página (SPA)
- ➔ Permite crear rutas y asociarlas a componentes, gestionando la navegación dentro de la aplicación

Instalación en proyecto existente básica

```
npm install vue-router
```



Vue Router. Configuración básica

En *app.vue*

```
<template>
  <router-view></router-view>
</template>
```

En *main.js*

```
import { createApp } from 'vue'
import { createWebHistory, createRouter } from 'vue-router'
import App from './App.vue'
import Home from './components/Home.vue'
import About from './components/About.vue'

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

createApp(App).use(router).mount('#app')
```



Instalación en creación de proyecto

→ Crear un proyecto Vue con NPM

```
npm create vue@latest
```

```
✓ Project name: ... <your-project-name>
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes ←
✓ Add Pinia for state management? ... No / Yes
✓ Add Vitest for Unit testing? ... No / Yes
✓ Add Cypress for both Unit and End-to-End testing? ... No / Yes
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes
Scaffolding project in ./<your-project-name>...
Done.
```



Vue Router. Navegación con RouterLink

- ➔ Vue Router incorpora el componente router-link para facilitar la navegación entre rutas registradas

```
<template>
  <div>
    <nav>
      <router-link to="/">Home</router-link>
      <router-link to="/about">About</router-link>
    </nav>
    <router-view></router-view>
  </div>
</template>
```



Vue Router. Rutas dinámicas

- ➔ Nos permite pasar parámetros a los componentes
- ➔ Por ejemplo, si quisiéramos mostrar la información con un determinado ID, crearemos una ruta como ***/users/123*** donde ***123*** es el identificador del usuario
- ➔ Los segmentos dinámicos se representan por dos puntos, ejemplo: ***/user/:id***
- ➔ el componente este segmento dinámico estará disponible con el composable ***useRoute***

```
...  
const router = createRouter({  
  history: createWebHistory(),  
  routes: [  
    { path: '/user/:id', component: User }  
  ]  
})  
...
```

```
<script setup>  
import { useRoute } from 'vue-router'  
  
const route = useRoute()  
</script>  
  
<template>  
  <div>User ID: {{ $route.params.id }}</div>  
</template>
```

patrón	ruta	useRoute
/user/:username/post/:post_id	/user/federico/post/123	{ username: 'federico', post_id: '123' }



Vue Router. Navegación programática

- ➔ La navegación programática permite cambiar la ruta desde el script en lugar de utilizar enlaces en el template

Métodos principales

- ➔ `router.push`: Navega a una nueva ruta.
- ➔ `router.replace`: Navega a una nueva ruta sin dejar una entrada en el historial.
- ➔ `router.go`: Navega a un número específico de pasos en el historial del navegador.

```
<template>
  <button @click="navigateToHome">Ir a Home</button>
</template>

<script setup>
import { useRouter } from 'vue-router'
const router = useRouter()
function navigateToHome() {
  router.push('/')
}
</script>
```



Vue Router. Guardas de navegación

- Las guardas de navegación permiten controlar el acceso a rutas específicas.
- Se pueden utilizar para verificar permisos, autenticar usuarios, o ejecutar lógica antes de que una navegación ocurra.

Tipos de guardas

- Guardas Globales: Aplicadas a todas las rutas.
- Guardas de Ruta: Definidas en rutas específicas.
- Guardas de Componente: Definidas dentro de componentes individuales.



Vue Router. Guarda global

```
<script setup>
import { createRouter, createWebHistory } from 'vue-router'
import Home from './components/Home.vue'
import About from './components/About.vue'

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

router.beforeEach((to, from, next) => {
  if (!isAuthenticated()) {
    next('/login')
  } else {
    next()
  }
})

function isAuthenticated() {
  // Lógica de autenticación
  return false
}

app.use(router)
</script>
```




Vue Router. Guarda de ruta

```
<script setup>
import { createRouter, createWebHistory } from 'vue-router'
import Home from './components/Home.vue'
import About from './components/About.vue'

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About, beforeEnter: (to, from, next) => {
    if (!isAuthenticated()) {
      next('/login')
    } else {
      next()
    }
  }}
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

function isAuthenticated() {
  // Lógica de autenticación
  return false
}

app.use(router)
</script>
```



Vue Router. Guarda de componente

```
<template>
  <div>
    <p>Contenido de About</p>
  </div>
</template>

<script setup>
import { onBeforeRouteLeave, onBeforeRouteUpdate } from 'vue-router'

onBeforeRouteLeave((to, from, next) => {
  const answer = window.confirm('¿Seguro que quieres salir?')
  if (answer) {
    next()
  } else {
    next(false)
  }
})

onBeforeRouteUpdate((to, from, next) => {
  // Manejar la actualización de la ruta
  next()
})
</script>
```



Vue Router. Guardas asíncronas

```
<script setup>
import { createRouter, createWebHistory } from 'vue-router'
import Home from './components/Home.vue'
import About from './components/About.vue'

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About }
]

const router = createRouter({
  history: createWebHistory(),
  routes,
})

router.beforeEach(async (to, from, next) => {
  try {
    const isAuthenticated = await checkAuth()
    if (to.meta.requiresAuth && !isAuthenticated) {
      next('/login')
    } else {
      next()
    }
  } catch (error) {
    next(error)
  }
})

async function checkAuth() {
  // Lógica de autenticación asíncrona
  return false
}

app.use(router)
</script>
```



Vue Router. Guardas con metadata

```
<script setup>
import { createRouter, createWebHistory } from 'vue-router'
import Home from './components/Home.vue'
import About from './components/About.vue'

const routes = [
  { path: '/', component: Home },
  { path: '/about', component: About, meta: { requiresAuth: true } }
]
const router = createRouter({
  history: createWebHistory(),
  routes,
})
router.beforeEach((to, from, next) => {
  if (to.meta.requiresAuth && !isAuthenticated()) {
    next('/login')
  } else {
    next()
  }
})
function isAuthenticated() {
  // Lógica de autenticación
  return false
}
app.use(router)
</script>
```



Pinia



¿Qué es Pinia?

- Sirve como **almacenamiento centralizado** para todos los componentes de una aplicación.
- Este almacenamiento centralizado es lo que se conoce como **estado de la aplicación**.
- Proporciona una manera **eficiente y reactiva** de administrar el estado de nuestra aplicación.



¿Cuándo usarlo?

- En aplicaciones medianas / grandes.
- Cuando necesitamos la misma información en distintas partes de la aplicación.
- Para reducir las peticiones a servicios externos.



Usando Pinia



Componente principal



Dashboard



Listado de noticias



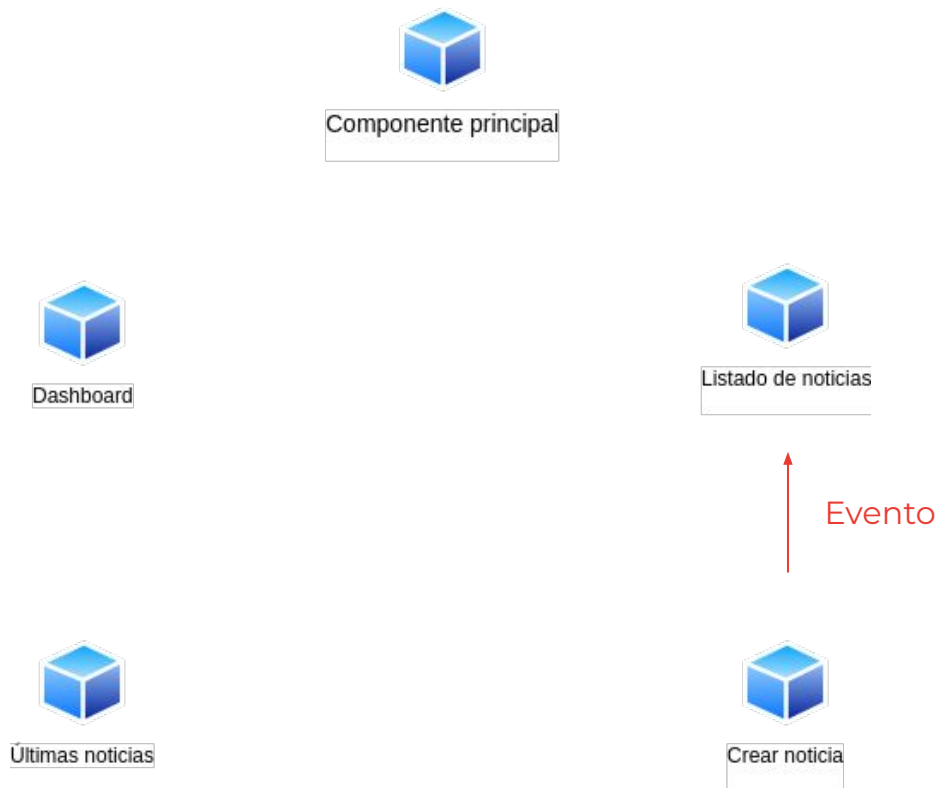
Últimas noticias



Crear noticia

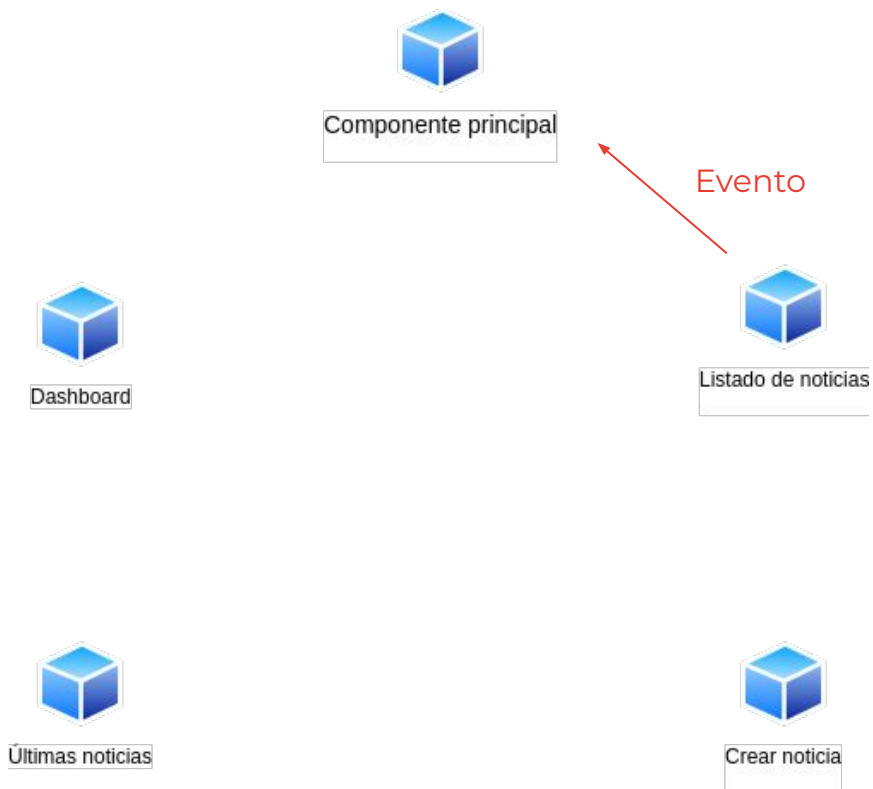


Usando Pinia



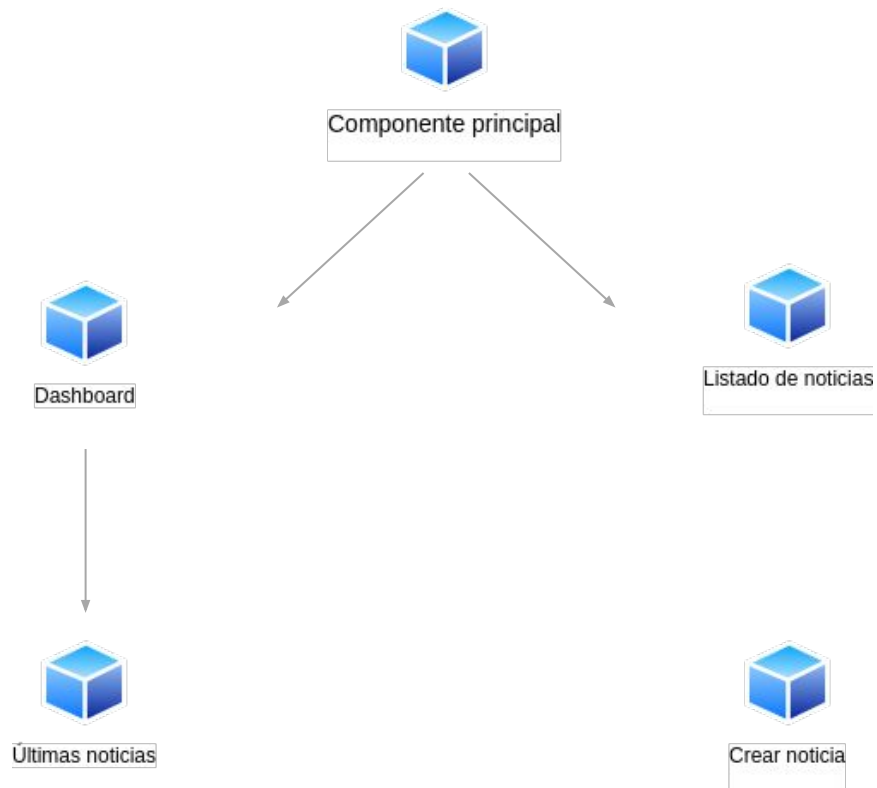


Usando Pinia





Usando Pinia





Usando Pinia



Componente principal



Dashboard



Listado de noticias

Actualizado



Últimas noticias

¡Actualizado!



Crear noticia



Usando Pinia



Store



Dashboard



Listado de noticias



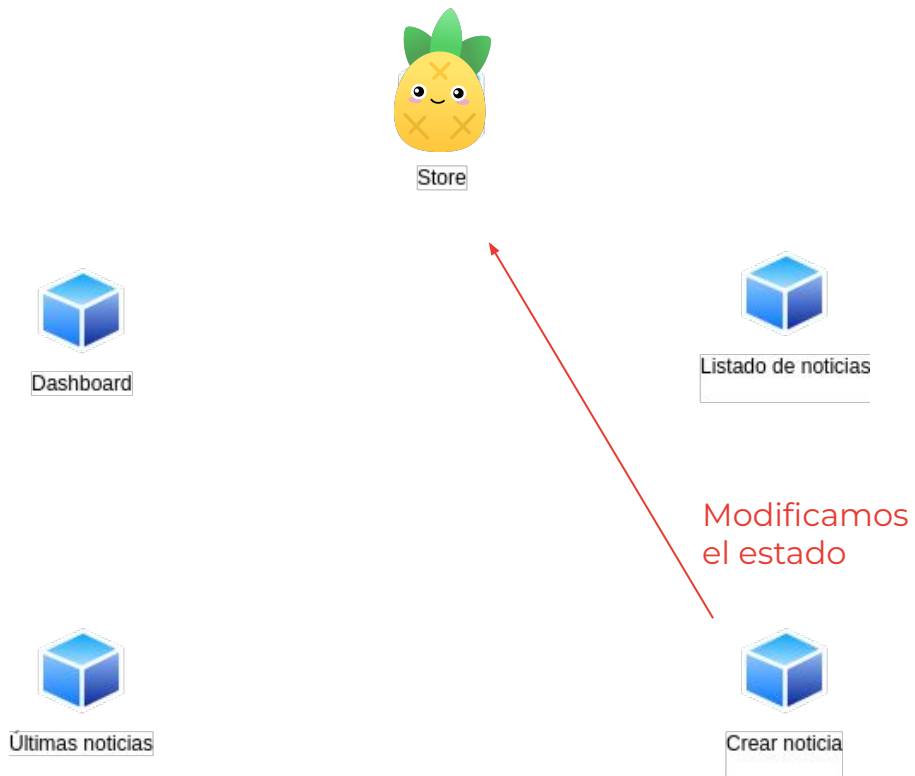
Últimas noticias



Crear noticia

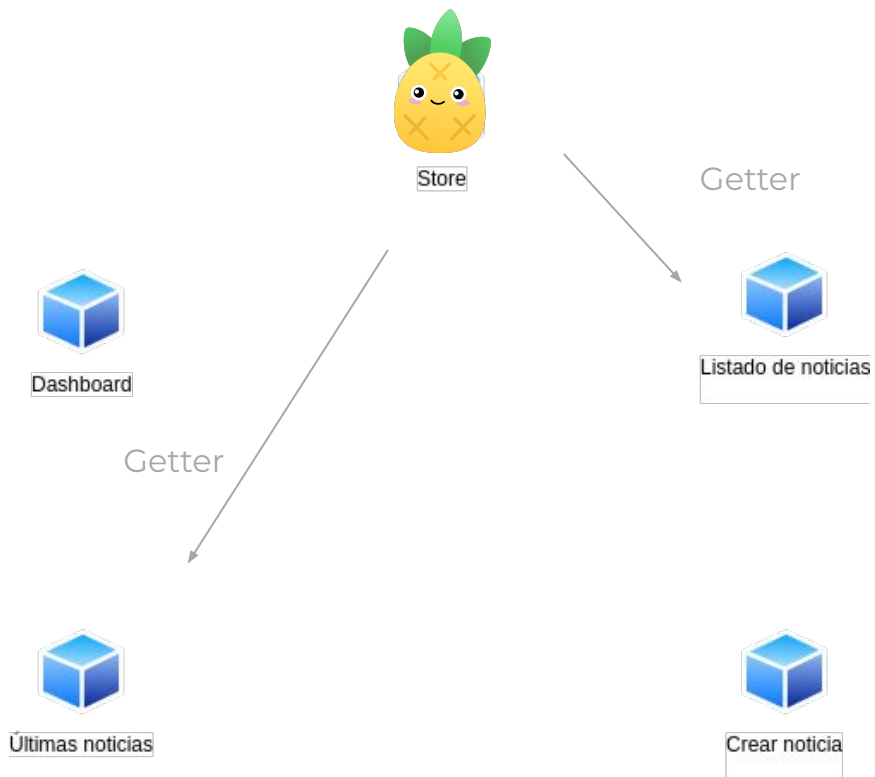


Usando Pinia





Usando Pinia





Instalación + configuración Pinia





Instalación de Pinia en un proyecto ya existente

→ Instalamos la dependencia y configuramos en el punto de entrada de VUE 3

```
npm install pinia
```

```
// main.js (punto entrada de nuestra app VUE)
```

```
import { createApp } from 'vue';  
import { createPinia } from 'pinia';  
import App from './App.vue';
```

```
const app = createApp(App);
```

```
// Configuramos Pinia con VUE (antes de montar la app)
```

```
const pinia = createPinia();  
app.use(pinia);
```

```
app.mount('#app');
```



Creación de un proyecto con Pinia

→ Crear un proyecto Vue con NPM

```
npm create vue@latest
```

```
✓ Project name: ... <your-project-name>
✓ Add TypeScript? ... No / Yes
✓ Add JSX Support? ... No / Yes
✓ Add Vue Router for Single Page Application development? ... No / Yes
✓ Add Pinia for state management? ... No / Yes ←
✓ Add Vitest for Unit testing? ... No / Yes
✓ Add Cypress for both Unit and End-to-End testing? ... No / Yes
✓ Add ESLint for code quality? ... No / Yes
✓ Add Prettier for code formatting? ... No / Yes
Scaffolding project in ./<your-project-name>...
Done.
```



Creación de un store con Pinia

→ Crear un proyecto Vue con NPM

```
// /store/authStore.js

import { defineStore } from 'pinia'

export const useAuthStore = defineStore('storeId', {
  // contenido del store
})
```



Conceptos principales

→ **state**

lugar donde almacenamos la toda la información. Puede modificarse a directamente o a través de acciones.

→ **actions**

Métodos que ejecutan lógica y guardan en el estado.
Ideadas para funciones asíncronas

→ **getters**

Métodos para acceder a las propiedades del state que requieren de una lógica previa (similar a un computed)



Crear el state

→ Los datos del state los añadiremos como variables reactiva que retornaremos

```
import { defineStore } from 'pinia'
import { ref } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const name = ref('Eduardo')
  const isAdmin = ref(true)

  return {
    count,
    name,
    isAdmin
  }
})
```



Uso de state

→ Para acceder a la propiedades del state las parsearemos con ***storeToRefs***

```
import { useCounterStore } from './stores/counter'  
import { storeToRefs } from 'pinia'  
  
const counterStore = useCounterStore()  
const { count, name } = storeToRefs(counterStore)
```

```
// ✗ No podemos pasarla directamente ya que no será reactiva  
const { count } = counterStore
```



Crear getters

- Se usan cuando necesitamos manipular los datos del state para realizar algún cálculo.
- Son computadas

```
import { defineStore } from 'pinia'
import { ref, computed } from 'vue'

export const useCounterStore = defineStore('counter', () => {
  const count = ref(0)
  const doubleCount = computed(() => count.value * 2)
  return {
    count,
    doubleCount
  }
})
```



Uso de getters

- Los computed expuestos por Pinia se consumen como propiedades
- NO hace falta .value fuera del store

```
import { useCounterStore } from './stores/counter'  
import { storeToRefs } from 'pinia'  
  
const counterStore = useCounterStore()  
const { doubleCount } = storeToRefs(counterStore)
```




Crear getters parametrizables

- Es posible pasar parámetros adicionales a nuestros getters (patrón currying)

```
// /stores/todoStore.js
...
getters: {
  // el getter devuelve una función ejecutable con los parámetros
  getTodoById: (state) => (id) => {
    return state.todos.find(todo => todo.id === id)
  }
}
...
```



Crear actions

→ Son funciones que exponemos en el store

```
export const useTodoStore = defineStore('todos', () => {
  const todos = ref([])
  async function loadTodos() {
    try {
      const response = await fetch(
        'https://jsonplaceholder.typicode.com/todos'
      )
      todos.value = await response.json()
    } catch (error) {
      console.error(error)
    }
  }
  return {
    todos,
    loadTodos
  }
})
```



Uso de actions

→ Creamos las acciones como funciones en la propiedad actions

```
import { useTodoStore } from './stores/todos'  
const todoStore = useTodoStore()  
await todoStore.loadTodos()
```



Práctica. Autenticación Simulada

Implementar una interfaz básica de autenticación donde un usuario puede iniciar sesión con un nombre de usuario. Usar Pinia para manejar el estado del usuario autenticado.

- Crear un store con estado inicial vacío para el usuario.
- Usar acciones para simular el inicio y cierre de sesión.
- Proteger rutas/componentes mostrando contenido condicional basado en el estado de autenticación.

Bonus

Haz una autenticación real con JWT (puedes usar AXIOs o fetch) usando [DummyJSON](#)